

Practica 1 – Automatizacion con Bash y Python: Captura y Visualizacion de Datos de Sensores

Autores:

Ander Zuazquita Pastor

Lucia Castellanos Paz



1.INTRODUCCION

Para esta practica se nos ha pedido desarrollar un sistema automatizado para la captura, almacenamiento y visualización de datos de sensores ambientales.

El sistema que nos dan en el enunciado solo mostraba los datos por la terminal sin almacenarlos, y nos pide encontrar una solución usando Bash y Python para automatizar todo el proceso de captura y análisis de los datos.

El sistema captura mensajes generados en formato JSON, los almacena en un archivo log y luego lo procesa para generar graficas.

2.OBJETIVOS

Los principales objetivos de la practica son:

- Entender el funcionamiento de la gestión de procesos
- Ejecutar el programa desde Bash y capturar la salida
- Redigir stdout y stderr a un archivo .log
- Gestionar procesos usando PID
- Verificar procesos activos con kill -0
- Finalizar procesos mediante señales
- Ejecutar Python desde bash
- Parsear logs y generar visualizaciones

3.MARCO TEORICO

PID: en un sistema operativo se pueden ejecutar varios procesos de manera concurrente, y cada proceso posee un identificador llamado PID que permite al sistema gestionar su estado.

&: en Bash es posible ejecutar un proceso en segundo plano gracias a este operador.

\$!: si un proceso se esta ejecutando en segundo plano, Bash nos deja obtener su PID mediante esta variable.

Kill -0: nos permite comprobar si un proceso sigue activo sin tener que enviar una señal real.

SIGTERM: es una señal que permite terminar un proceso de forma controlada.

Matplotlib: es una biblioteca de Python que genera graficas a partir de datos numéricos.

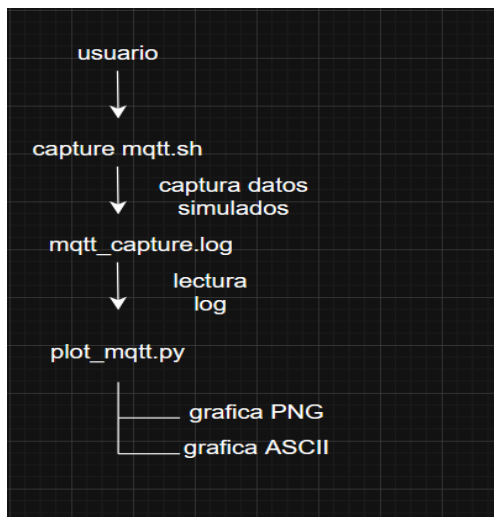
4.DISEÑO Y METODOLOGIA

El sistema esta compuesto de dos partes importantes y repartidas entre ambos.

- Un script Bash que se encarga de la captura de datos y gestión de procesos.
- Un script Python encargado del análisis y visualización de los datos.

Tras realizar los git pull correspondientes cada uno empezó con la implementación y creación del Bash y Python. Ambos tuvimos una serie de problemas a lo largo de su creación, pero conseguimos el siguiente flujo del sistema:

1. El usuario ejecuta el script bash e introduce el tiempo de captura
2. El script inicia un proceso que genera datos simulados de los sensores
3. Estos datos se almacenan en el archivo mqtt_capture.log
4. Al finalizar el tiempo indicad, el proceso se detiene
5. El script ejecuta el programa de Python para que se analicen los datos
6. Python genera una grafica PNG y su representación ASCII en la terminal



Para que se generara tanto la grafica como su representación ASCII tuvimos que usar el siguiente comando para instalar matplotlib:

sudo apt install python3-matplotlib

Al instalarse con escribir python3 y ya dentrole Python escribir:

import matplotlib

print("Todo funciona")

```
lucia@luu:~/practical_so/practical-so$ python3
Python 3.12.3 (main, Jan 22 2026, 20:57:42) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

y con esto comprobamos que python funcionaba correctamente.

5.IMPLEMENTACION

SCRIPT BASH

El script capture_mqtt.sh se encarga del proceso de captura de datos, controlar la ejecución y ejecutar Python.

Este proceso se ejecuta en segundo plano utilizando & , y el PID del proceso se obtiene con \$!

```
GNU nano 7.2 capture_mqtt.sh
#!/bin/bash
echo "script de captura MQTT"
echo "iintroduce el tiempo de captura en segundos:"
read tiempo #solicita el tiempo y lo guarda en la variable tiempo
echo "iniciando captura de datos..."
while true; do #bucle infinito con la recepcion de datos
echo "Payload: {\\"AmbientTemperature\\":${((RANDOM % 40))}}"; sleep 1; done >
#Random genera un numero aleatorio y %40 limita el numero entre 0 y 39
#done> mqtt_capture.log &: guarda la salida en mqtt_capture.log y ejecuta e
PID=$! #guarda el PID del proceso en segundo plano para controlarlo
echo "Proceso iniciado con PID: $PID" #muestra el identificador del proceso
contador=0 #controla el tiempo de captura
while kill -0 $PID 2>/dev/null #comprueba si el proceso sigue activo usando
do
sleep 1 #espera 1 segundo antes de generar el siguiente dato
contador=$((contador+1)) #incrementa el contador

if [ $contador -ge $tiempo ]; then #comprueba si el tiempo de captura h
echo "Tiempo alcanzado. Finalizando proceso..."
kill -SIGTERM $PID #envia la señal SIGTERM para finalizar el proceso
break #sale del bucle
fi
done
echo "Ejecutando analisis en Python..."
python3 plot_mqtt.py #ejecuta el script de pythom que analiza los datos y g
```

SCRIPT PYTHON

El script plot_mqtt.py se encarga de leer el archivo de log generado por el Bash

```
GNU nano 7.2 plot_mqtt.py *
import json #importa la libreria json para poder leer datos JSON
import matplotlib.pyplot as plt #importa matplotlib para generar graficas
archivo="mqtt_capture.log" #nombre del archivo log generado por el Bash
with open(archivo, "r") as f: #abre el archivo log en modo lectura
    lineas=f.readlines() #lee todas las lineas del archivo y las guarda en
datos=[] #lista para guardar los datos JSON del log
for linea in lineas: #recorre todas las lineas
    if "Payload:" in linea: #comprueba si la linea contiene datos del sensor
        json_str=linea.split("Payload:")[1] #extrae la parte JSON del mensa
        try: #para evitar errores
            data=json.loads(json_str) #convierte el JSON en un dic de python
            datos.append(data) #añade el dato extraido a la lista de datos
        except: #ignora errores
            pass

print("numero de mensajes encontrados:",len(datos)) #muestra cuantos mensaje
if len(datos)==0: #comprueba si no se encontraron datos
    print("no se encontraron datos en el log.")
    print("asegurate de ejectar primero el script Bash para capturar datos")
    exit() #termina el programa si no hay datos

temperaturas = [] #lista para almacenar las temperaturas
for d in datos:
    if "AmbientTemperature" in d: #comprueba si el dato contiene temperatura
        temperaturas.append(d["AmbientTemperature"]) #guarda el valor de te
plt.plot(temperaturas) #genera una grafica con los valores temperatura
plt.title("temperatura del sensor")
plt.xlabel("tiempo") #etiqueta eje X
plt.ylabel("temperatura") #etiqueta eje Y
plt.savefig("plots/temperatura.png") #guarda la grafica en la carpeta plots
print("graficaa guardada en plots/temperatura.png")

print("\ngrafica ASCII:\n") #muestra una representacion ASCII de la grafica
for t in temperaturas:
    barras=int(t) #convierte la temperatura en numero de caracteres
    print("|" + "#" * barras) #genera barras con ASCII
```

La imagen de debajo muestra la ejecución del script Python encargado de proesar el archivo de log y la generación de una grafica que se guarda en la carpeta plots.

```
lucia@luu:~/practical1_so/practical1-so$ python3 plot_mqtt.py
numero de mensajes encontrados: 0
graficaa guardada en plots/temperatura.png
lucia@luu:~/practical1_so/practical1-so$ |
```

Durante la prueba se verifiko que el script Python podía abrir el archivo de log generado por el script bash

```
lucia@luu:~/practical1_so/practical1-so$ nano plot_mqtt.py
lucia@luu:~/practical1_so/practical1-so$ python3 plot_mqtt.py
Archivo cargado correctamente
lucia@luu:~/practical1_so/practical1-so$ |
```

6.RESULTADOS Y PRUEBAS

Durante las pruebas se ejecuto el sistema con diferentes tiempos de captura:

5segundos:

```
script de captura MQTT
introduce el tiempo de captura en segundos:
5
iniciando captura de datos...
Proceso iniciado con PID: 1441
Tiempo alcanzado. Finalizando proceso...
Ejecutando analisis en Python...
numero de mensajes encontrados: 6
graficaa guardada en plots/temperatura.png
```

grafica ASCII:

```
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
```

10segundos:

```
anderzp in @anderHP in :~/practical1-so $ ./capture_mqtt.sh
script de captura MQTT
introduce el tiempo de captura en segundos:
10
Iniciando captura de datos...
Proceso iniciado con PID: 4353
Tiempo alcanzado. Finalizando proceso...
Ejecutando analisis en Python...
numero de mensajes encontrados: 10
qt.qpa.plugin: Could not find the Qt platform plugin "wayland" in ""
graficaa guardada en plots/temperatura.png
```

grafica ASCII:

```
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
|#####|
```

```
anderzp in @anderHP in :~/practical1-so $ |
```

15segundos:

[illegible]

Las pruebas muestran que el sistema es capaz de capturar correctamente los datos, almacenarlos en un archivo de log y generar las visualizaciones.

Problemas: A la hora de instalar el programa de MQTT que nos pasas en el archivo instal.sh para visualizarlo, nos daba varios errores de que no encontraba mensajes y por tanto tuvimos que simular los sensores con Payload y para que no funcionara nada, optamos por hacer eso. No sabemos a que se debe el error si era por el script de Bash o Python y lo pasamos varias veces por chatgpt pero no daba resultados.

7.DISCUSION

Durante la realización de la practica encontramos algunas dificultades relacionadas con la gestión de procesos y la generación de datos para las pruebas.

Como mencionamos anteriormente intentamos usar el cliente MQTT real pero por problemas que desconocemos no compilaba bien y optamos por simular los datos de sensores usando valores aleatorios.

A pesar de ello conseguimos realizar todas las pruebas necesarias y comprobar el funcionamiento del sistema.

README fue redactado por nosotros pero la estructura para que se vea bien, fue realizada por chatgpt como se indica en el archivo, ya que al escribirlo se veía bastante mal visualmente.

Como mejora intentaríamos usar el bróker MQTT real para capturar los datos y aprender a presentar y estructurar el README por nosotros mismos.

8.CONCLUSIONES

En esta practica hemos sidpo capaces de desarrollar un sistema para la captura y análisis de datos utilizando Bash y Python.

Además hemos usado herramientas de análisis de datos de Python para generar representaciones graficas a partir de información capturada.

9.MANUAL DE EJECUCION

Para ejecutar esta practica es necesario disponer de Python3 y la biblioteca Matplotlib instalada.

Pasos:

Dar permisos al script Bash:

chmod +x capture_mqtt.sh

Ejecutar el script:

./capture_mqtt.sh

Introduces el tiempo deseado y una vez finalizado el proceso se genera la grafica en la carpeta plots.

10.REPOSITORIO GIT

Enlace a nuestro repositorio Git:

<https://github.com/lukks775/practica1-so>

Estructura del proyecto:

plots	grafica de temperatura generada por python	2 days ago
README.md	Estructura inicial del proyecto	2 weeks ago
capture_mqtt.sh	comentarios explicativos script bash	2 hours ago
informe.pdf	Estructura inicial del proyecto	2 weeks ago
mqtt_capture.log	grafica de temperatura generada por python	2 days ago
plot_mqtt.py	comentarios explicativos script python	1 hour ago

11.BIBLIOGRAFIA

Python3:

<https://python-guide-es.readthedocs.io/es/latest/starting/install3/linux.html>

Matplotlib:

<https://agrportfolioeducativo.blogspot.com/2016/05/47-instalando-modulo-matplotlib-en.html>

Git:

<https://docs.github.com/es/repositories/working-with-files/managing-files/adding-a-file-to-a-repository>